

9 ПРАКТИЧЕСКАЯ РАБОТА (ПОЛЬЗОВАТЕЛИ, РОЛИ И РАЗГРАНИЧЕНИЕ ДОСТУПА)

9.1 Цель работы

Целью практической работы является формирование навыков проектирования и реализации модели разграничения доступа к данным и программным объектам реляционной базы данных посредством пользователей, ролей и привилегий, а также проверки корректности разработанной модели на основании пользовательских сценариев.

9.2 Задачи работы

В процессе выполнения практической работы необходимо:

- проанализировать пользовательские роли, определённые на предыдущих этапах проектирования;
- определить информационные ресурсы и операции, доступные каждой роли;
- сформировать матрицу разграничения доступа;
- определить набор привилегий, необходимый для каждой роли;
- создать роли средствами выбранной СУБД;
- создать тестовых пользователей либо иным способом обеспечить проверку работы ролей;
- предоставить необходимые права посредством механизмов СУБД;

- ограничить выполнение запрещённых операций;
- определить права на таблицы, представления, функции и процедуры;
- проверить корректные сценарии доступа;
- проверить сценарии отказа в доступе;
- убедиться в невозможности обхода разработанной модели посредством избыточных привилегий;
- оформить модель доступа в виде воспроизводимого SQL-скрипта;
- подготовить ролевую модель к использованию в графическом пользовательском интерфейсе.

9.3 Теоретические сведения

Разграничение доступа представляет собой совокупность правил и механизмов, определяющих, какие пользователи или категории пользователей могут выполнять конкретные действия над информационными ресурсами системы.

В системе управления данными разграничение доступа может распространяться на:

- чтение данных;
- добавление данных;
- изменение данных;
- удаление данных;
- выполнение функций;
- выполнение хранимых процедур;
- использование представлений;

- создание и изменение объектов базы данных;
- выполнение административных операций.

Основная задача разграничения доступа заключается в предоставлении пользователю только тех полномочий, которые необходимы для выполнения его функций.

Не рекомендуется предоставлять всем пользователям одинаковый полный доступ к базе данных, поскольку это противоречит самой идее ролевого управления и существенно снижает защищённость системы.

9.3.1 Пользователь и роль

Необходимо различать понятия пользователя и роли.

Пользователь представляет конкретную учётную запись, посредством которой выполняется подключение к системе управления базами данных.

Роль представляет совокупность полномочий, соответствующих определённой категории пользователей или функции в системе.

Например, в системе могут быть определены роли:

- администратор;
- оператор;
- менеджер;
- аналитик;
- сотрудник;
- клиент.

При этом несколько различных пользователей могут обладать одной ролью.

Использование ролей позволяет не назначать полный набор привилегий каждому пользователю отдельно.

Вместо этого:

- создаётся роль;
- роли назначаются необходимые права;
- пользователю назначается соответствующая роль.

Такой подход упрощает сопровождение модели доступа.

9.3.2 Ролевая модель доступа

Ролевая модель доступа предполагает предоставление полномочий не непосредственно конкретным пользователям, а на основании их функций в системе.

Упрощённо модель может быть представлена следующим образом:

пользователь → роль → набор разрешённых операций → объекты системы.

Например:

Оператор

может:

- просматривать заявки;
- создавать заявки;
- изменять определённые сведения;
- назначать исполнителя посредством предусмотренной процедуры;

но не может:

- управлять пользователями;
- изменять структуру базы данных;
- удалять журнал истории.

Аналитик

может:

- просматривать необходимые данные;
- выполнять аналитические запросы;
- получать отчёты;

но не должен иметь возможность изменять операционные данные.

Таким образом, различные роли должны обладать действительно различающимися полномочиями.

9.3.3 Принцип минимально необходимых привилегий

При проектировании модели доступа рекомендуется применять принцип минимально необходимых привилегий.

Согласно данному принципу, пользователь должен получать только те права, которые необходимы ему для выполнения предусмотренных функций.

Например, если пользователю требуется только просматривать данные, ему не должны автоматически предоставляться права:

- INSERT;
- UPDATE;
- DELETE.

Если изменение данных должно выполняться исключительно через специальную хранимую процедуру, предоставление прямого права UPDATE на соответствующую таблицу может сделать разработанное ограничение бессмысленным.

Принцип минимально необходимых привилегий позволяет:

- снизить вероятность случайного повреждения данных;
- ограничить последствия ошибочных действий;

- уменьшить возможность обхода бизнес-логики;
- сделать модель доступа более контролируемой.

9.3.4 Привилегии

Конкретный перечень привилегий зависит от используемой СУБД.

Для объектов реляционной базы данных могут использоваться права, соответствующие операциям:

- чтения;
- вставки;
- изменения;
- удаления;
- выполнения серверного объекта;
- создания объектов;
- использования отдельных элементов схемы.

Наиболее распространёнными привилегиями над таблицами являются:

- SELECT;
- INSERT;
- UPDATE;
- DELETE.

Для функций и процедур может использоваться право выполнения, например EXECUTE, если оно поддерживается выбранной СУБД.

Синтаксис и модель прав различаются между СУБД, поэтому конкретные команды необходимо выбирать в соответствии с используемой платформой.

9.3.5 Предоставление привилегий

Операция предоставления права обычно выполняется посредством команды GRANT.

Общая логика операции:

```
GRANT <привилегия>  
ON <объект>  
TO <пользователь или роль>;
```

Например, концептуально роль может получить:

- право чтения таблицы;
- право изменения определённых данных;
- право выполнения процедуры.

Конкретный синтаксис зависит от выбранной СУБД.

Перед предоставлением привилегии необходимо определить:

- кому она предоставляется;
- на какой объект;
- для выполнения какой функции системы;
- действительно ли необходим прямой доступ к объекту.

9.3.6 Отзыв привилегий

Для удаления ранее предоставленного права используется механизм REVOKE.

Общая логика:

```
REVOKE <привилегия>  
ON <объект>  
FROM <пользователь или роль>;
```

Использование REVOKE позволяет проверить не только предоставление доступа, но и возможность его последующего изменения.

В рамках настоящей практической работы обучающийся должен продемонстрировать понимание обоих механизмов.

9.3.7 Права на таблицы

Права могут назначаться непосредственно на таблицы.

Например, для определённой роли может быть разрешено:

- чтение таблицы;
- добавление строк;
- изменение строк;
- удаление строк.

Однако предоставление прямого доступа к таблице не всегда является оптимальным решением.

Если сложная бизнес-операция реализована процедурой, более корректной моделью может быть:

- запрет прямого изменения таблицы;
- разрешение выполнения соответствующей процедуры.

Такой подход позволяет уменьшить возможность обхода серверной логики.

9.3.8 Права на функции и процедуры

Разработанные в ПР №8 функции и процедуры должны быть учтены при проектировании модели доступа.

Необходимо определить:

- какие роли могут использовать соответствующий объект;
- какие роли не должны иметь доступа;
- требуется ли пользователю дополнительно прямой доступ к используемым таблицам;
- каким образом конкретная СУБД определяет контекст выполнения серверного объекта.

Например, если процедура реализует операцию:

назначить исполнителя,

то право её выполнения должно предоставляться только той роли, для которой соответствующая функция предусмотрена техническим заданием.

Предоставление права всем пользователям без анализа недопустимо.

9.3.9 Триггеры и разграничение доступа

Триггеры не вызываются пользователем непосредственно.

Однако пользовательские операции могут приводить к их срабатыванию.

Следовательно, при проектировании модели доступа необходимо учитывать:

- кто имеет право выполнять исходную операцию;
- какие действия автоматически выполняются триггером;
- не позволяет ли предоставленная привилегия косвенно нарушить установленное бизнес-правило.

Например, если история изменения состояния создаётся триггером, пользователь может иметь право изменить состояние объекта, но не обязательно должен иметь право непосредственно изменять таблицу истории.

9.3.10 Матрица доступа

До написания SQL-команд необходимо спроектировать модель доступа на логическом уровне.

Для этого рекомендуется использовать матрицу доступа.

Пример:

Объект / операция	Администратор	Оператор	Аналитик
Просмотр клиентов	+	+	+
Добавление клиентов	+	+	—
Изменение клиентов	+	+	—
Удаление клиентов	+	—	—
Просмотр заявок	+	+	+
Изменение статуса	+	+	—
Выполнение аналитической функции	+	—	+
Управление пользователями	+	—	—

Обозначения:

- + — операция разрешена;
- – — операция запрещена.

При необходимости матрица может быть более детализированной.

Например, вместо общего действия «изменение заказа» могут отдельно определяться:

- изменение состава;
- изменение состояния;
- отмена;
- подтверждение.

9.3.11 Различие прикладной и административной роли

Необходимо различать прикладные роли пользователей системы и техническую роль администратора базы данных.

Администратор СУБД объективно может обладать расширенными полномочиями.

Однако наличие такой административной учётной записи не отменяет необходимости реализации прикладного разграничения доступа.

Минимальное требование:

не менее двух ролей

относится именно к содержательно различающимся прикладным ролям.

Например:

- admin;
- superadmin;

не считаются достаточной парой ролей, если обе обладают практически одинаковыми полномочиями.

Аналогично использование:

- владелец базы;
- обычный пользователь;

не заменяет анализ ролей предметной области.

9.3.12 Связь ролей с требованиями системы

Каждая прикладная роль должна иметь основание в результатах предыдущих практических работ.

Следует проверить цепочку:

роль ПР №1 → требования ПР №2 → Use Case ПР №4 → права ПР №9.
--

Например:

если в техническом задании указано:

оператор может создавать и изменять заявки, но не может управлять пользователями,

то модель доступа должна технически обеспечивать это различие.

Недостаточно просто скрыть кнопку управления пользователями в будущем GUI.

Если пользователь имеет прямые полномочия на выполнение запрещённой операции в базе данных, модель доступа является неполной.

9.3.13 Разграничение доступа и пользовательский интерфейс

В последующей ПР №10 будет разработан графический пользовательский интерфейс.

GUI должен учитывать модель доступа, разработанную в настоящей работе.

Это означает, что интерфейс должен:

- определять права текущего пользователя;
- предоставлять доступ только к разрешённым операциям;
- не отображать или блокировать недоступные действия;
- корректно обрабатывать отказ СУБД в выполнении операции.

При этом интерфейс не должен быть единственным уровнем защиты.

Скрытие кнопки:

«Удалить пользователя»

не является разграничением доступа, если та же учётная запись имеет DELETE на соответствующую таблицу.

9.3.14 Ошибка как нормальный результат проверки доступа

При тестировании модели разграничения доступа часть операций должна завершаться отказом.

Например:

роль «Аналитик» выполняет попытку удалить запись.

Ожидаемый результат:

операция отклонена СУБД вследствие отсутствия необходимой привилегии.

Такая ошибка является **корректным результатом тестового сценария**.

Цель проверки состоит не в том, чтобы все SQL-команды завершались успешно, а в том, чтобы разрешённые действия выполнялись, а запрещённые — блокировались.

9.4 Задание

На основании ролей, требований, физической структуры и серверной логики проекта необходимо разработать и реализовать модель разграничения доступа.

Работа должна включать:

- анализ прикладных ролей;
- матрицу доступа;
- создание ролей средствами СУБД;
- назначение привилегий;
- ограничение запрещённых операций;
- проверку разработанной модели.

9.4.1 Анализ пользовательских ролей

Необходимо сформировать итоговый перечень прикладных ролей на текущем этапе проекта.

Для каждой роли следует указать:

- назначение;
- основные пользовательские функции;
- объекты данных, с которыми она работает;
- разрешённые операции;
- запрещённые операции.

Роль должна иметь реальное функциональное отличие от других ролей.

9.4.2 Формирование матрицы доступа

Необходимо сформировать матрицу доступа, содержащую как минимум:

- основные таблицы или логические группы данных;
- существенные функции и процедуры;
- основные операции;
- прикладные роли.

Для каждой комбинации необходимо определить разрешение или запрет.

Матрица доступа должна быть согласована с техническим заданием.

9.4.3 Создание ролей

Необходимо создать не менее 2 содержательно различающихся прикладных ролей средствами выбранной СУБД.

При необходимости могут быть созданы дополнительные роли:

- административная;
- служебная;
- роль только для чтения;
- иные роли, необходимые архитектуре проекта.

Дополнительные технические роли не заменяют минимальные две прикладные роли.

9.4.4 Создание тестовых пользователей

Для проверки модели рекомендуется создать не менее одной тестовой учётной записи для каждой прикладной роли либо использовать иной штатный механизм выбранной СУБД, позволяющий выполнить проверку прав роли.

Каждая тестовая учётная запись должна использоваться исключительно в учебных целях.

Не следует включать реальные персональные пароли в материалы отчёта или репозитория проекта.

9.4.5 Реализация привилегий

Необходимо назначить роли права в соответствии с разработанной матрицей.

При проектировании следует рассмотреть права:

- SELECT;
- INSERT;
- UPDATE;
- DELETE;
- выполнение функций;
- выполнение процедур;
- иные необходимые привилегии выбранной СУБД.

Не требуется использовать каждую разновидность привилегии для каждой роли.

Права должны определяться реальными пользовательскими функциями.

9.4.6 Использование GRANT

В скрипте должен быть продемонстрирован механизм предоставления привилегий.

Как минимум для нескольких объектов должны присутствовать операции GRANT либо их функциональный эквивалент в выбранной СУБД.

Студент должен быть способен объяснить каждое предоставляемое право.

Формулировка:

«выдал все права, чтобы работало»

не является допустимым обоснованием.

9.4.7 Использование REVOKE

В рамках работы необходимо продемонстрировать механизм REVOKE либо его функциональный эквивалент.

Это может быть реализовано, например, посредством:

- предварительного предоставления права;
- его последующего отзыва;
- проверки невозможности выполнения соответствующей операции.

Либо REVOKE может использоваться для явного удаления наследуемого или ранее назначенного права в рамках разработанной модели.

9.4.8 Ограничение прямого доступа

Необходимо проанализировать серверные операции, разработанные в ПР №8.

Если бизнес-операция должна выполняться исключительно через процедуру, следует рассмотреть модель:

разрешить выполнение процедуры → запретить непосредственное изменение соответствующих таблиц,
--

если используемая СУБД и архитектура проекта позволяют реализовать такой подход.

Подобное решение должно быть обосновано.

9.5 Минимальные требования к результату

Для зачёта практической работы необходимо обеспечить:

- не менее 2 содержательно различающихся прикладных ролей;
- матрицу разграничения доступа;
- различающиеся наборы полномочий ролей;
- использование механизма GRANT;
- использование механизма REVOKE либо эквивалентного механизма отзыва прав;
- разграничение доступа как минимум к таблицам и серверным объектам, если последние используются проектом;
- проверку не менее 4 содержательных сценариев доступа;
- наличие как разрешённых, так и запрещённых сценариев;
- воспроизводимый SQL-скрипт создания ролей и назначения прав.

Минимальное количество ролей не должно достигаться путём создания нескольких ролей с одинаковыми полномочиями.

9.6 Требования к сценариям проверки

Необходимо разработать не менее четырёх сценариев проверки.

В минимальный набор должны входить:

- разрешённая операция чтения;
- разрешённая операция изменения либо выполнения серверного объекта;
- запрещённая операция изменения данных;

- запрещённая операция, характерная для другой роли.

Например:

№	Роль	Действие	Ожидаемый результат
1	Оператор	Просмотреть заявки	Разрешено
2	Оператор	Назначить исполнителя	Разрешено
3	Аналитик	Удалить заявку	Отказ
4	Оператор	Управлять пользователями	Отказ

Для проекта с несколькими ролями рекомендуется расширить данный перечень.

9.6.1 Проверка права выполнения функции или процедуры

Если в ПР №8 реализованы пользовательские функции или процедуры, необходимо проверить как минимум один сценарий доступа к серверному объекту.

Например:

- роль А может выполнять процедуру;
- роль Б не может выполнять эту процедуру.

Если архитектура выбранной СУБД требует дополнительных прав на используемые объекты, соответствующая особенность должна быть отражена в отчёте.

9.6.2 Проверка отсутствия избыточных прав

После реализации модели необходимо проверить, что роли не получили дополнительных полномочий вследствие:

- членства в другой роли;

- привилегий по умолчанию;
- предоставления чрезмерно широких прав;
- подключения под владельцем базы данных;
- использования общей привилегированной учётной записи.

Особое внимание следует уделить ситуации, когда приложение в дальнейшем подключается к БД от имени одной общей учётной записи.

Если такая учётная запись имеет все права, а ролевое разграничение реализовано только внутри GUI, фактическая модель доступа на уровне СУБД может не соответствовать заявленной.

9.6.3 SQL-скрипт ролей и прав

Разработанная модель должна быть представлена воспроизводимым SQL-скриптом.

Рекомендуемый файл:

```
06_roles.sql
```

При сохранении ранее принятой нумерации файлов допускается использование 05_roles.sql.

Главное требование — единообразная последовательность файлов во всём проекте.

Скрипт должен содержать:

- создание ролей;
- необходимые настройки ролей;
- предоставление прав;
- отзыв прав;

- при необходимости — создание тестовых пользователей;
- назначение пользователям ролей.

9.7 Рекомендуемая структура SQL-каталога

После завершения настоящей работы структура может иметь следующий вид:

```
/project
  /sql
    01_create_tables.sql
    02_insert_data.sql
    03_queries.sql
    04_functions.sql
    05_procedures.sql
    06_triggers.sql
    07_roles.sql
```

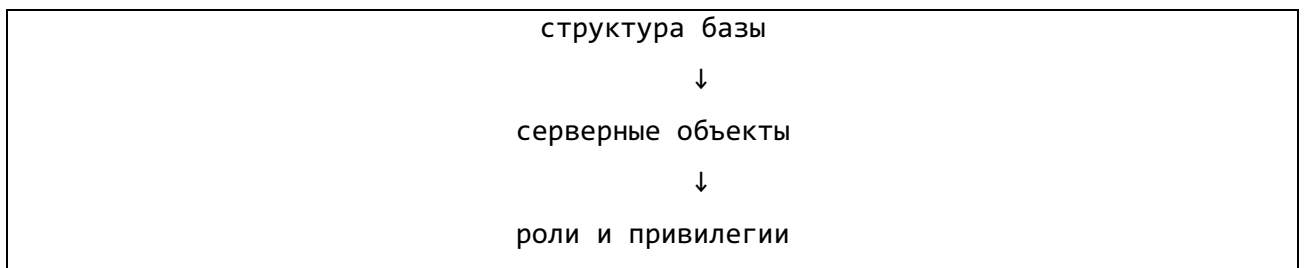
Если функции и процедуры объединены в один файл, нумерация может отличаться.

Во всём проекте должна использоваться одна согласованная схема именования.

9.8 Воспроизводимость модели доступа

Необходимо обеспечить возможность развернуть модель доступа после создания основных объектов базы данных.

Порядок должен учитывать зависимости:



Если права выдаются на процедуру или функцию, соответствующий объект должен существовать к моменту выполнения команды предоставления привилегии.

9.9 Проверка согласованности с предыдущими практическими работами

Перед завершением работы необходимо проверить:

- роли соответствуют ПР №1;
- полномочия соответствуют требованиям ПР №2;
- акторы UML соответствуют прикладным ролям либо имеют объяснимое отличие;
- операции соответствуют бизнес-процессам;
- права назначены на существующие физические объекты;
- функции и процедуры из ПР №8 учтены в модели доступа;
- нет серверной операции, доступной роли, которой эта операция запрещена требованиями;
- изменение модели ролей отражено в техническом задании.

9.10 Порядок выполнения работы

- Проанализировать замечания по ПР №8.
- Уточнить итоговый перечень пользовательских ролей.
- Составить перечень объектов, требующих разграничения.
- Определить разрешённые действия для каждой роли.
- Определить запрещённые действия.
- Сформировать матрицу доступа.
- Проверить её соответствие техническому заданию.
- Создать роли средствами СУБД.
- Создать тестовые учётные записи либо подготовить иной механизм проверки.
- Назначить разрешённые привилегии.
- Выполнить необходимые операции REVOKE.
- Назначить права на функции и процедуры.
- Проверить разрешённый сценарий чтения.
- Проверить разрешённый сценарий изменения.
- Проверить запрещённый сценарий изменения.
- Проверить запрещённую операцию другой роли.
- Проверить доступ к серверным объектам.
- Проверить отсутствие избыточных прав.
- Проверить воспроизводимость SQL-скрипта.
- Актуализировать предыдущие материалы при необходимости.
- Подготовить отчёт.
- Подготовить систему к прохождению контрольной точки №3.

9.11 Содержание отчёта

Отчёт по практической работе должен содержать:

- титульный лист;
- номер и название практической работы;
- цель работы;
- перечень прикладных ролей;
- описание назначения каждой роли;
- матрицу разграничения доступа;
- перечень создаваемых пользователей или способ проверки ролей;
- основные команды создания ролей;
- примеры GRANT;
- примеры REVOKE;
- описание прав на функции и процедуры;
- перечень сценариев проверки;
- ожидаемые результаты;
- фактические результаты;
- примеры корректно отклонённых операций;
- анализ отсутствия избыточных полномочий;
- сведения об изменениях предыдущих материалов;
- вывод по результатам работы.

9.12 Результаты, предоставляемые преподавателю

Для защиты практической работы обучающийся предоставляет:

- отчёт;
- актуальные SQL-скрипты проекта;
- скрипт создания ролей и назначения прав;
- матрицу доступа;
- тестовую базу либо возможность её воспроизвести.

При защите обучающийся должен быть готов:

- объяснить назначение каждой роли;
- показать различие полномочий ролей;
- выполнить подключение или переключение контекста для проверки роли;
- продемонстрировать разрешённую операцию;
- продемонстрировать запрещённую операцию;
- объяснить причину отказа;
- предоставить или отозвать выбранную привилегию;
- объяснить, почему конкретное право необходимо пользователю;
- показать разграничение доступа к функции или процедуре;
- объяснить, каким образом модель будет использоваться в GUI.

9.13 Использование результатов в курсовой работе

Модель разграничения доступа является частью итоговой архитектуры системы управления данными.

Для зачёта практической работы достаточно:

- двух содержательно различающихся прикладных ролей;
- установленного минимального набора проверок.

Для курсовой работы количество ролей должно определяться фактическим составом пользователей итоговой системы.

Если предметная область предусматривает:

- администратора;
- оператора;
- руководителя;
- аналитика;
- клиента,

ограничение модели только двумя ролями исключительно потому, что этого было достаточно для ПР №9, будет рассматриваться как неполная реализация требований.

В курсовой работе необходимо обеспечить согласованность:

пользовательские роли → функциональные требования → UML → полномочия БД → GUI → тестирование.

Каждая существенная роль должна быть отражена:

- в описании системы;
- в техническом задании;
- в модели вариантов использования;
- в модели доступа;
- в пользовательском интерфейсе;
- в тестовых сценариях.

9.14 Требования к GUI в курсовой работе

На следующем этапе ролевое разграничение должно быть интегрировано в приложение.

Необходимо избегать архитектуры, в которой:

- все пользователи работают через одну привилегированную учётную запись;
- все ограничения реализуются только посредством скрывания элементов интерфейса;
- прямой доступ к базе позволяет выполнять запрещённые операции.

Для курсовой работы необходимо показать, каким образом реализованная ролевая модель учитывается приложением.

В зависимости от архитектуры это может быть реализовано различными способами, однако выбранный подход должен быть объясним и согласован с моделью безопасности базы данных.

9.15 Последствия выполнения работы только на минимальном уровне

Если в ПР №9 реализовать две формальные роли, практически не соответствующие реальным пользователям системы, на этапе создания GUI придётся переработать:

- сценарии авторизации;
- доступные экраны;
- действия;
- SQL-права;

- права на процедуры;
- тестовые сценарии.

Следовательно, студентам, использующим проект как основу курсовой работы, рекомендуется уже в настоящей ПР реализовывать предполагаемый итоговый набор основных ролей.

9.16 Контрольная точка №3. Готовность ядра системы

Успешная защита практической работы №9 завершает формирование ядра системы управления данными.

К моменту прохождения контрольной точки №3 должны быть защищены и согласованы:

- ПР №5 — логическая модель данных;
- ПР №6 — физическая структура БД;
- ПР №7 — тестовые данные и SQL-запросы;
- ПР №8 — функции, процедуры и триггеры;
- ПР №9 — пользователи, роли и права доступа.

После прохождения контрольной точки система должна обладать работоспособным серверным уровнем, который может использоваться графическим пользовательским интерфейсом.

До начала полноценной разработки GUI должны быть стабильны:

- структура данных;
- основные запросы;
- серверная бизнес-логика;
- модель доступа.

9.16.1 Условия прохождения контрольной точки №3

Контрольная точка считается пройденной при выполнении следующих условий:

- ПП №6–9 защищены;
- база данных разворачивается из SQL-скриптов;
- тестовые данные загружаются без необходимости ручной корректировки;
- основные SQL-запросы выполняются;
- функции и процедуры доступны предусмотренным ролям;
- триггеры работают в соответствии с требованиями;
- ролевая модель технически реализована;
- разрешённые операции выполняются;
- запрещённые операции блокируются;
- материалы проекта взаимно согласованы.

При наличии критических противоречий переход к окончательной разработке GUI может быть отложен до их устранения.

9.17 Вспомогательные материалы

Для каждой роли рекомендуется сформировать карточку:

Параметр	Значение
Наименование роли	
Назначение	
Основные пользовательские функции	
Доступные таблицы	
Разрешённые операции	
Доступные функции	
Доступные процедуры	
Запрещённые действия	

Для проверки каждого права рекомендуется задать вопросы:

- Из какого требования следует необходимость этого права?
- Можно ли предоставить более узкое право?
- Должен ли пользователь изменять таблицу непосредственно?
- Можно ли выполнять операцию только через процедуру?
- Что произойдёт при отсутствии этого права?
- Не позволяет ли данное право выполнить запрещённую операцию?

9.18 Контрольные вопросы

- В чём заключается назначение разграничения доступа?
- Чем пользователь отличается от роли?
- В чём заключается ролевая модель доступа?
- Что понимается под принципом минимально необходимых привилегий?
- Для чего применяется GRANT?

- Для чего применяется REVOKE?
- Какие основные права могут предоставляться на таблицы?
- Почему не рекомендуется предоставлять всем пользователям полный доступ?
- Для чего разграничивается право выполнения функций и процедур?
- Почему триггер обычно не требует прямого вызова пользователем?
- Что представляет собой матрица доступа?
- Почему административная учётная запись не заменяет прикладные роли?
- Как требования системы связаны с моделью доступа?
- Почему скрывание элемента GUI не является полноценным механизмом ограничения доступа?
- Почему запрещённая операция должна входить в тестирование модели доступа?
- В каких случаях прямое изменение таблицы может быть запрещено при наличии процедуры?
- Какие проблемы возникают при подключении приложения под привилегированной учётной записью?
- Как модель доступа должна быть связана с последующей ПР по GUI?
- Что означает принцип отсутствия избыточных прав?
- Какие элементы должны быть готовы к контрольной точке №3?

9.19 Критерии приёмки работы

Практическая работа может быть зачтена при одновременном выполнении следующих условий:

- определены не менее двух содержательных прикладных ролей;
- роли имеют различающиеся полномочия;
- сформирована матрица доступа;
- права соответствуют техническому заданию;
- реализован механизм GRANT;
- продемонстрирован механизм REVOKE;
- права на таблицы определены осознанно;
- права на функции и процедуры учтены при их наличии;
- проверено не менее четырёх содержательных сценариев;
- присутствуют разрешённые сценарии;
- присутствуют запрещённые сценарии;
- запрещённые операции действительно блокируются СУБД;
- отсутствует необоснованное предоставление полных прав;
- модель реализована воспроизводимым SQL-скриптом;
- студент способен объяснить каждую существенную привилегию;
- студент способен предоставить и отозвать право;
- модель согласована с ПР №1–8;
- отсутствуют роли, созданные исключительно ради выполнения количественного минимума.

После успешной защиты ПР №9 и выполнения установленных условий считается пройденной контрольная точка №3.

9.20 Срок выполнения и сдачи

Срок выполнения и защиты практической работы определяется календарным графиком практических занятий.

Практическая работа №9 должна быть завершена до начала полноценной разработки графического пользовательского интерфейса.

К моменту перехода к ПР №10 должен существовать работоспособный серверный уровень системы, включающий структуру данных, тестовые данные, запросы, серверную бизнес-логику и технически реализованную модель разграничения доступа.